OPERATING SYSTEMS
MINI PROJECT REPORT

FILE MANAGEMENT SYSTEM

A simple menu-driven program to create, read, rename and delete files

Student Name:	Basudev Tiwari
Roll Number:	25ET2051013
Branch:	B.Tech CSE
Subject:	Operating Systems
College:	Alard University

Academic Year: 2026–2027

Certificate

This is to certify that the mini project titled **“File Management System”** has been prepared as part of the Operating Systems coursework. The project demonstrates basic file operations through a menu-driven program.

Acknowledgement

I would like to thank my subject teacher for the guidance and opportunity to complete this mini project. I also thank my college and classmates for their support. This project helped me understand how programs can use file-handling operations to store and manage information.

Abstract

The File Management System is a beginner-friendly, menu-driven C++ application that allows a user to create a text file, write content, read a file, rename a file, and delete a file. It uses standard C++ file streams and basic file-system functions. The project provides a practical introduction to persistent storage and file operations that are commonly used by applications running on an operating system.

Table of Contents

• 1. Introduction

• 2. Problem Statement

• 3. Objectives

• 4. Scope and Requirements

• 5. System Design / Algorithm

• 6. Source Code

• 7. How to Compile and Run

• 8. Sample Output and Testing

• 9. Advantages and Limitations

• 10. Conclusion and Future Scope

• 11. References

1. Introduction

A file management system helps users perform basic operations on files stored on a computer. Files are used to keep information even after a program is closed. In this mini project, a C++ program provides a simple menu to create, view, rename, and delete text files. It is a simulation of common file-management tasks; it does not replace the operating system's own file manager.

2. Problem Statement

Managing files through separate commands can be confusing for beginners. The aim is to develop a simple program that presents common file operations in one menu and gives a clear message after each operation.

3. Objectives

• Create a new text file and save user-entered content.

• Read and display the contents of an existing file.

• Rename a file using a new filename.

• Delete a file after the user confirms the action.

• Understand basic C++ file streams and file-system operations.

4. Scope and Requirements

Hardware requirements

• A desktop or laptop computer.

• At least 2 GB RAM (typical basic development environment).

Software requirements

• C++ compiler such as GCC / MinGW or an IDE such as Code::Blocks or Visual Studio Code with a compiler.

• Operating system: Windows, Linux, or macOS.

Language: C++ (standard library).
Input: Menu choice, filenames, and text content.
Output: Success/error messages and file contents.

5. System Design / Algorithm

Main algorithm

• Start the program.

• Display the menu: create/write, read, rename, delete, or exit.

• Accept the user's choice.

• Perform the selected operation and display the result.

• Repeat the menu until the user selects Exit.

• Stop.

Pseudocode

```
START
REPEAT
    DISPLAY menu
    INPUT choice
```

```
    IF choice = 1: create/write file
    ELSE IF choice = 2: read file
    ELSE IF choice = 3: rename file
    ELSE IF choice = 4: delete file after confirmation
    ELSE IF choice = 5: exit
    ELSE: display invalid choice
UNTIL choice = 5
STOP
```

The program uses **ofstream** to write to a file, **ifstream** to read a file, and standard file-system functions to rename or remove a file.

6. Source Code

Save the following code as **file_management.cpp**. It is designed for a simple beginner-level console application.

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstdio>
#include <limits>
using namespace std;

void createFile() {
    string filename, content;
    cout << "Enter filename (example: notes.txt): ";
    getline(cin, filename);
    ofstream file(filename);
    if (!file) {
        cout << "Error: Could not create/open the file.\n";
        return;
    }
    cout << "Enter content (one line): ";
    getline(cin, content);
    file << content << '\n';
    file.close();
    cout << "File saved successfully.\n";
}

void readFile() {
    string filename, line;
    cout << "Enter filename to read: ";
    getline(cin, filename);
    ifstream file(filename);
    if (!file) {
        cout << "Error: File not found or cannot be opened.\n";
        return;
    }
    cout << "\n--- File Content ---\n";
    while (getline(file, line)) cout << line << '\n';
    cout << "----------------------\n";
}

void renameFile() {
    string oldName, newName;
    cout << "Enter current filename: ";
    getline(cin, oldName);
    cout << "Enter new filename: ";
    getline(cin, newName);
    if (rename(oldName.c_str(), newName.c_str()) == 0)
        cout << "File renamed successfully.\n";
    else
        cout << "Error: Rename failed. Check filenames and permissions.\n";
}

void deleteFile() {
    string filename, answer;
    cout << "Enter filename to delete: ";
    getline(cin, filename);
    cout << "Are you sure you want to delete '" << filename << "'? (yes/no): ";
    getline(cin, answer);
    if (answer == "yes" || answer == "YES" || answer == "Yes") {
        if (remove(filename.c_str()) == 0)
            cout << "File deleted successfully.\n";
        else
            cout << "Error: Delete failed. Check filename and permissions.\n";
    } else {
        cout << "Delete cancelled.\n";
    }
}
```

```
int main() {
    int choice = 0;
    do {
        cout << "\n===== FILE MANAGEMENT SYSTEM =====\n";
        cout << "1. Create / Write a file\n";
        cout << "2. Read a file\n";
        cout << "3. Rename a file\n";
        cout << "4. Delete a file\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        if (!(cin >> choice)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid input. Enter a number from 1 to 5.\n";
            continue;
        }
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        switch (choice) {
            case 1: createFile(); break;
            case 2: readFile(); break;
            case 3: renameFile(); break;
            case 4: deleteFile(); break;
            case 5: cout << "Exiting program. Goodbye!\n"; break;
            default: cout << "Invalid choice. Try again.\n";
        }
    } while (choice != 5);
    return 0;
}
```

Safety note: Test using sample files in a dedicated project folder. The delete option removes the selected file, so do not enter the name of an important file.

7. How to Compile and Run

• Create a project folder and save the code as file_management.cpp.

• Open a terminal or command prompt in that folder.

• Compile with GCC: `g++ file_management.cpp -o file_management`.

• Run on Windows: `file_management.exe`; on Linux/macOS: `./file_management`.

• Use sample filenames such as notes.txt while testing.

8. Sample Output

```
===== FILE MANAGEMENT SYSTEM =====
1. Create / Write a file
2. Read a file
3. Rename a file
4. Delete a file
5. Exit
Enter your choice: 1
Enter filename (example: notes.txt): notes.txt
Enter content (one line): This is my OS mini project.
File saved successfully.

Enter your choice: 2
Enter filename to read: notes.txt

--- File Content ---
This is my OS mini project.
--------------------
```

Test cases

Test	Action	Expected result
1	Create notes.txt	File is created and content is saved.
2	Read notes.txt	Saved content is displayed.
3	Rename notes.txt to memo.txt	Filename changes if operation succeeds.

Test	Action	Expected result
4	Read a missing file	A clear error message is displayed.
5	Cancel deletion	File remains unchanged.

9. Advantages and Limitations

Advantages

• Simple menu-driven interface.

• Demonstrates create, read, rename, and delete operations.

• Uses standard C++ libraries and is suitable for beginners.

• Can be tested with small text files.

Limitations

• Only one line of text is entered when creating a file in this basic version.

• No login, permissions management, recycle bin, or graphical interface.

• File operations depend on the current working directory and user permissions.

10. Conclusion and Future Scope

The File Management System mini project demonstrates fundamental file operations in C++. It provides practical exposure to persistent storage, input validation, and error handling. Future improvements could include multi-line editing, file listing, search, copy operations, a graphical interface, and safer confirmation or backup features.

11. References

• Bjarne Stroustrup, Programming: Principles and Practice Using C++.

• cplusplus.com — C++ file streams (ifstream and ofstream) and standard library reference.

• cppreference.com — C++ standard library reference.

— End of Report —